



Callbacks vs Promises vs Async/Await

Last Updated : 2 Sep, 2025

Handling asynchronous operations is an essential part of JavaScript programming, especially when dealing with APIs, file handling, database queries, and user interactions. JavaScript provides three main ways to manage asynchronous tasks:

- **Callbacks:** The traditional approach using functions passed as arguments.
- **Promises:** A better alternative that improves readability and avoids callback nesting.
- **Async/Await:** A modern and cleaner syntax that makes asynchronous code look synchronous.

Callbacks vs Promises vs Async/Await

Feature	Callbacks	Promises	Async/Await
Definition	Functions passed as arguments to other functions.	Objects representing eventual completion or failure of an asynchronous operation.	Syntactic sugar over Promises, using <code>async</code> functions and <code>await</code> expressions.
Readability	Can lead to "callback hell" with nested functions, making code hard to read.	Allows chaining with <code>.then()</code> and <code>.catch()</code> , improving readability.	Provides a more synchronous-like flow, enhancing readability.
Error Handling	Requires manual checks and can be cumbersome in nested callbacks.	Errors are handled using <code>.catch()</code> , centralizing error management.	Utilizes <code>try...catch</code> blocks, simplifying error handling.
Control Flow	Sequential execution, but complex flows can become difficult to manage.	Supports chaining, allowing for more manageable control flows.	Enables writing asynchronous code in a synchronous manner, simplifying control flow.
Support	Supported in all JavaScript environments.	Supported in all modern JavaScript environments.	Supported in ES2017 and later; may require transpilation for older environments.
Performance	Minimal overhead; direct function calls.	Slight overhead due to Promise object creation.	Similar to Promises; may have slight overhead but generally negligible.

What is Callback?

[Callbacks](#) are functions passed as arguments to other functions and are executed once a specific task is completed. They are commonly used in [JavaScript](#) for handling asynchronous operations but can lead to "[callback hell](#)" when nested multiple times.

```
// Callback function to process user data
function process(data, callback) {
  console.log("Processing user data:", data);
  callback();
}

// Function to fetch user data with a callback
function fetch(callback) {
  // Simulating an API request
  const user = { id: 1, name: "Pushkar" };
  callback(user);
}

// Using the callback
fetch((data) => {
  process(data, () => {
```

× ▶ 📄

JS Tutorial Web Tutorial A to Z Guide Projects OOP DOM Set Map Math Number Boolean Exercise 🔍 Sign In

```
});
```

Output

```
Processing user data: { id: 1, name: 'Pushkar' }
User data processed successfully.
```

In this example

- Callbacks are passed as arguments to other functions.
- They execute once the asynchronous operation is completed.
- Callback hell can occur with multiple nested callbacks, making the code harder to read and maintain.

What is Promises?

[Promises](#) offer a more structured approach to handle asynchronous operations, addressing the callback hell problem. They represent the eventual completion (or failure) of an asynchronous task.

```
// Function to fetch user data with a Promise
function fetch() {
  // Simulating an API request
  const user = { id: 1, name: "Pushkar" };

  return new Promise((resolve, reject) => {
    if (user) {
      resolve(user);
    } else {
      reject("Error fetching user data.");
    }
  });
}

// Using the Promise
fetch()
  .then((data) => {
    console.log("Processing user data:", data);
```

× ▶ 📄

```
        console.log("User data processed successfully.");
    })
    .catch((error) => {
        console.error("Error:", error);
    });
});
```

Output

```
Processing user data: { id: 1, name: 'Pushkar' }
User data processed successfully.
```

In this example

- Promises represent the completion of an asynchronous task.
- They are chained with `.then()` for successful completion and `.catch()` for errors.
- Promises improve readability compared to callbacks, preventing nested structures.

What is Async/Await?

[Async/Await](#) is built on top of Promises and allows asynchronous code to be written in a synchronous style, making it easier to read and understand.

```
// Function to fetch user data with Async/Await
async function fetch() {
    // Simulating an API request
    const user = { id: 1, name: "Pushkar" };

    return new Promise((resolve) => {
        resolve(user);
    });
}

// Function to process user data using Async/Await
async function process() {
    try {
        const data = await fetch();
        console.log("Processing user data:", data);
        console.log("User data processed successfully.");
    } catch (error) {
        console.error("Error:", error);
    }
}

// Using Async/Await
process();
```

Output

```
Processing user data: { id: 1, name: 'Pushkar' }
User data processed successfully.
```

In this example

- Async/Await simplifies asynchronous code, making it appear synchronous.
- The `await` keyword pauses the execution until the Promise is resolved.
- Async/Await is ideal for handling multiple asynchronous operations in a clear, easy-to-read way.

When to Use Callbacks

Callbacks are suitable for simple asynchronous operations where you only need to handle one or two asynchronous tasks. They are the original way to handle asynchronous code in JavaScript.

- **Simple Asynchronous Tasks:** When you have a single asynchronous operation to perform, callbacks can be straightforward.
- **Legacy Code:** You might encounter callbacks in older JavaScript codebases or libraries.

When to Use Promises

Promises are a better choice for managing more complex asynchronous code, especially when you have multiple operations that depend on each other.

- **Chaining Asynchronous Operations:** Promises excel when you need to execute asynchronous tasks sequentially.
- **Improved Error Handling:** Promises provide a cleaner way to handle errors compared to callbacks.
- **Modern JavaScript APIs:** Many modern JavaScript APIs and libraries are promise-based.

When to Use Async/Await

async/await is generally the preferred way to work with asynchronous code in modern JavaScript. It makes asynchronous code look and behave a bit more like synchronous code.

- **Readability and Maintainability:** async/await significantly improves the readability of complex asynchronous code.
- **Simplified Control Flow:** async/await allows you to use familiar control flow structures (like if statements and loops) within your asynchronous code.
- **Working with Promises:** async/await is built on top of promises, so it works seamlessly with promise-based APIs. It's generally the best approach when dealing with promises.

Similarity between Callbacks, Promises, and Async/Await:

- All these three are designed to handle the results of asynchronous operations. i.e. similarities in asynchronous operations and handling of asynchronous results.
- They all handle errors in their own way, Callbacks use the error first convention, Promises uses the .catch() method, and Async/Await uses try and catch block.
- Callbacks, Promises, and Async/Await they all operate on the JavaScript event loop. i.e. similarity in event loop utilization.

Suggested Quiz

🔄 4 Questions

Which of the following is the main disadvantage of using callbacks for asynchronous operations?

- ☐ (A) They cannot handle asynchronous tasks
- ☐ (B) They always run synchronously
- ☐ (C) They can cause deeply nested code, reducing readability
- ☐ (D) They are slower than Promises in all cases

Comment

A anilku... + Follow

5

Article Tags:

JavaScript

Web Technologies

Geeks Premier League

Geeks Premier League 2023

+1 More

Explore

JavaScript Basics

Array & String

Function & Object

OOP

Asynchronous JavaScript

Exception Handling

DOM

Advanced Topics



Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)



Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise with us
GFG
Corporate Solution
Training Program

Explore

POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core
Subjects
Interview Preparation
Software and Tools

Courses

ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design

